

Fundamental concepts

One of the main targets of multiprogramming is to maximize the use of the CPU resource. To reach this target, more than one task (i.e. process thread) is assigned to each program:

- The scheduler must implement the better scheduling algorithm for the assign of the CPU to a task
- Cost functions evaluate scheduler performance
- Different applications require different algorithms and cost functions

Algorithm

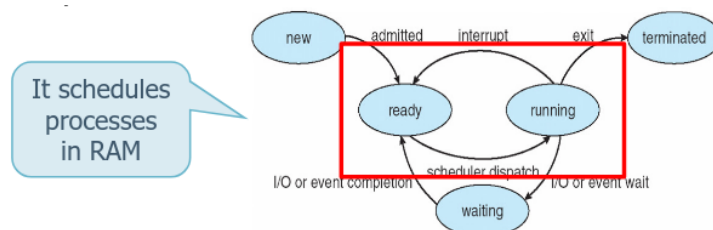
The general scheduling procedure is:

- CPU is assigned to a task
- Each time a process enters a waiting state, terminates, an interrupt is received, etc., it is necessary to perform a context switching operation
- For each context switching:
 - The task in the running state is moved in the ready queue
 - A task in the ready queue is moved in the running state

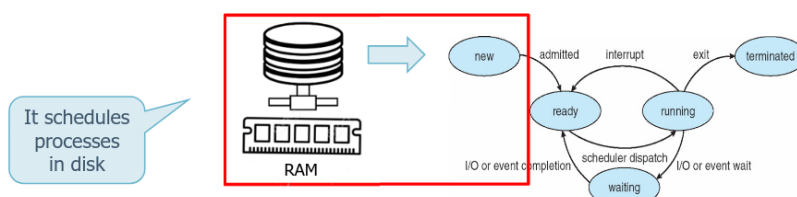
CPU scheduler

Several types exist:

- Short-term scheduler
 - Selects the process to which assign the CPU within the set of processes ready for execution in the main memory
 - Is executed very frequently
 - Is executed with timings in the order of milliseconds after an interrupt due to a timer or an I/O operation
 - Must be very fast

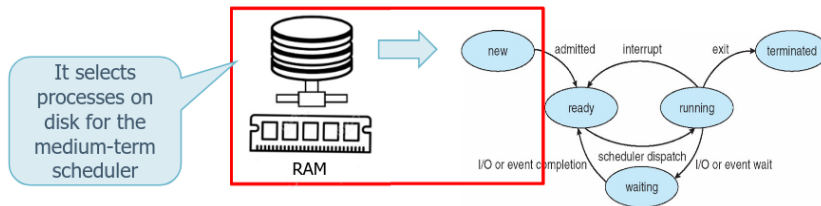


- Medium-term scheduler
 - Moves processes from the main memory to the second memory and vice versa
 - Selects which process to insert in the ready list
 - Is executed less frequently
 - Is executed with timings in the order of seconds
 - In practice, it controls the number of processes in RAM



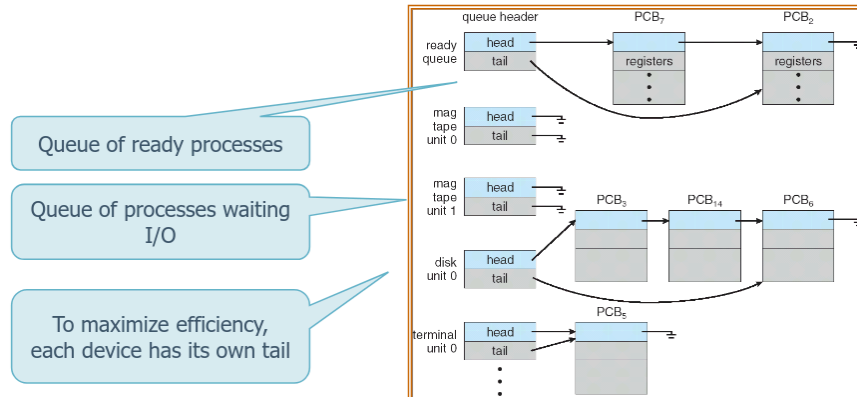
- Long-term scheduler
 - Also called job scheduler or admission scheduler selects the programs the system wishes to process based on its resources
 - Is executed much less frequently
 - Is executed with timings in the order of minutes

- It basically controls the degree of multiprogramming



The scheduler manages the processes waiting for a device through (process) queue:

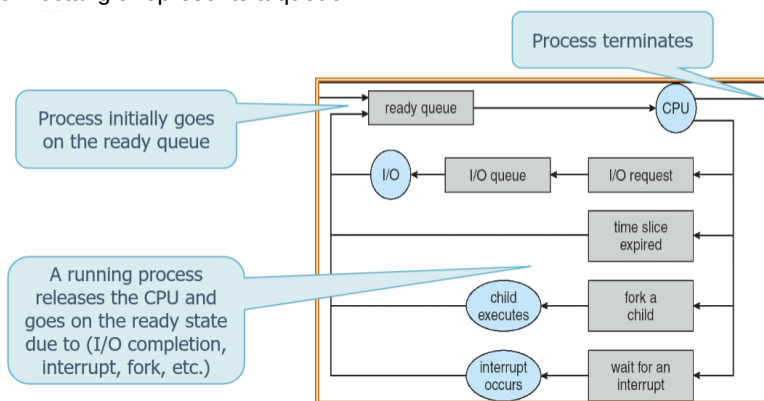
- There are several queues, one per device
- Each queue is a linked list



Queuing diagram

The queuing diagram shows the possible process transitions from one queue to another one:

- each rectangle represents a queue



Important algorithms

Algorithms

Extension

Algorithms without preemption

FCFS (First Come First Served)
Scheduling in order of arrival

SJF (Shortest Job First)
Scheduling in order of length

PS (Priority Scheduling)
Scheduling in order of priority

MQS (Multilevel Queue Scheduling) Multi-level queues scheduling

Algorithms with preemption

RR (Round Robin)
Circular scheduling

SRTF (Shortest Remaining Time
First)
Scheduling for minimum remaining
time

Non preemptive

The CPU is **not** subtracted to
another task, i.e., the task must
release the CPU voluntarily

Preemptive

The CPU can be subtracted to another
task, i.e., CPU burst are defined (e.g.,
maximum execution times) at the end
of which the CPU is reassigned to
another task

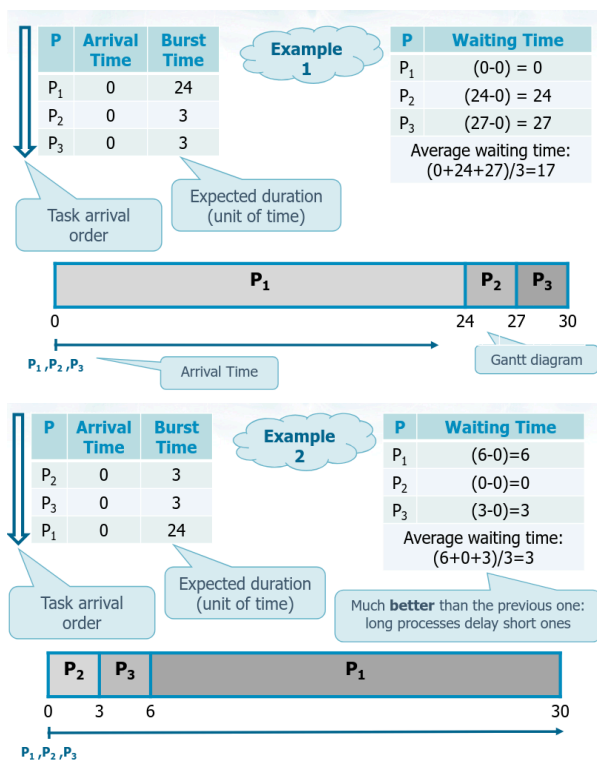
Cost function	Description	Optimum
CPU utilization	Percentage of CPU utilization	[0-100%] Maximum
Throughput	Number of processes completed in a time unit	Maximum
Turnaround time	Time that passes from the submission to the termination of a process	Minimum
Waiting time	Total time spent in the ready queue (sum of the times spent in the queue)	Minimum
Response time	Time elapsed between the submission and the production of the first response	Minimum

FCFS

Algorithm

The CPU is assigned to the tasks following the order in which they requested it:

- Tasks are managed through a FIFO queue
 - A new task is inserted in the queue tail
 - A task to serve is extracted from the queue head
- Scheduling can be sketched by means of a Gantt diagram
 - Bar chart showing the planning (start and end times) of the activities (Remember that no task is interrupted, i.e., the CPU can only be released voluntarily)

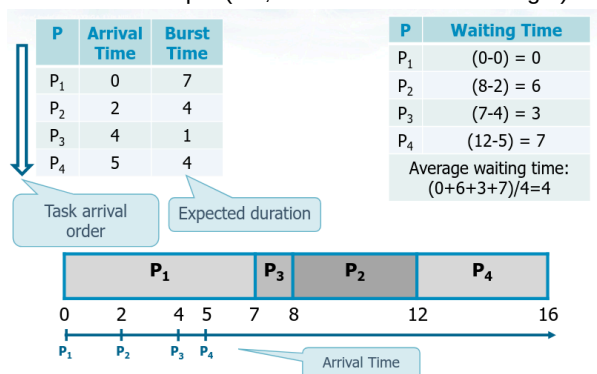


- Advantages
 - Easy to understand
 - Easy to implement
- Disadvantages
 - Waiting times
 - Relatively long
 - Variables and not optimal
 - Unsuitable for real-time systems (no preemption)
 - Queue effect
 - Short tasks queued after long tasks, wait for a long time uselessly

SJF

Algorithm

- To each task is associated the duration of the next CPU request (next CPU burst)
- The tasks are scheduled in order of duration of their next request
 - Scheduling in order of length
- In case of ex-aequo (i.e., tasks with the same length) the FCFS scheduling is applied



- Advantages
 - It can be demonstrated that SJF is an optimal algorithm, using the waiting time as a criterion
 - By moving the short processes before the long ones, the waiting time of the first decreases more than the increase of the waiting time of the seconds
- Disadvantages
 - Possible starvation

- Difficult of application, due to the impossibility to know a priori the future behavior of the task
 - Next burst time is unknown
 - It is possible to estimate this time using different methods (e.g., the exponential average)

Exponential average:

$$\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \tau_n$$

Where:

τ_{n+1} : Expected value for the next burst

$\alpha \cdot t_n$: (Real) duration of the n-th burst

$\alpha = [0, 1]$: controls the relative weight recent vs past history

$$\alpha = 0 \rightarrow \tau_{n+1} = \tau$$

$$\alpha = 1 \rightarrow \tau_{n+1} = t_n$$

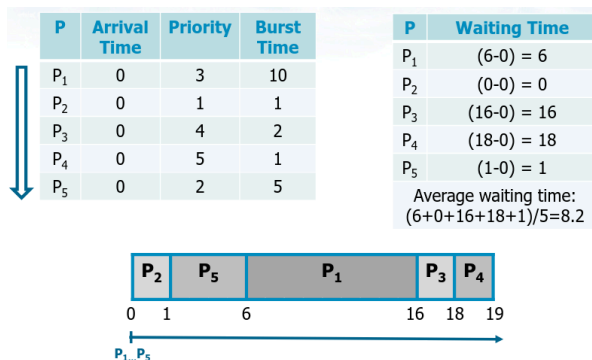
Proceeding by substitution:

$$\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \alpha \cdot t_{n-1} + \dots + (1 - \alpha)^j \cdot \alpha \cdot t_{n-j} + \dots + (1 - \alpha)^{n+1} \cdot \tau_0$$

Since both α and $1 - \alpha$ are minor than 1, older terms weight less

PS

- A priority associated to each task
 - Priority is typically represented with integer number
 - The higher the priority the smaller the integer number
 - Priorities can be determined based on
 - Internal criteria: used memory, number of used files, etc.
 - External criteria: owner of the task, etc.
- CPU is allocated to the task with higher priority
 - PS = SJF with the duration of the CPU burst substituted with the priority



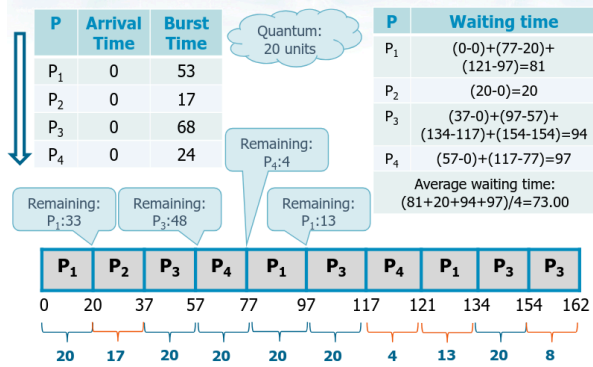
- Drawbacks
 - Possible starvation
 - In highly loaded systems, tasks with low priority can wait forever
 - MIT: IBM stopped in 1973 with a process queued since 1967
 - A possible solution to starvation is aging of tasks
 - The tasks priority is gradually increase over time

RR

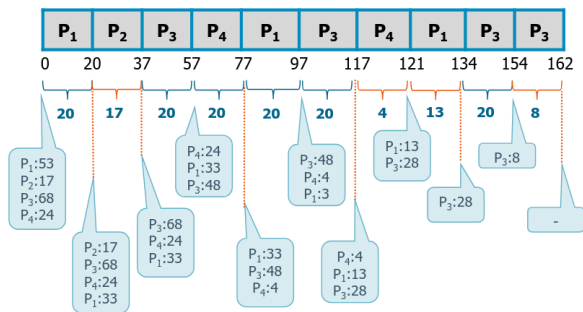
Round Robin or circular scheduling. Is a version of FCFS with preemption.

- Algorithm
 - The CPU usage is divided into "time quantum" (i.e., discrete temporal intervals)
 - Each task can use the CPU for a maximum time equal to the quantum, and then it is inserted again in the ready queue
 - The ready queue is managed using a FIFO policy
 - New processes are inserted in the ready queue

- Designed specifically for time sharing (and some basic real-time systems)



❖ More in details

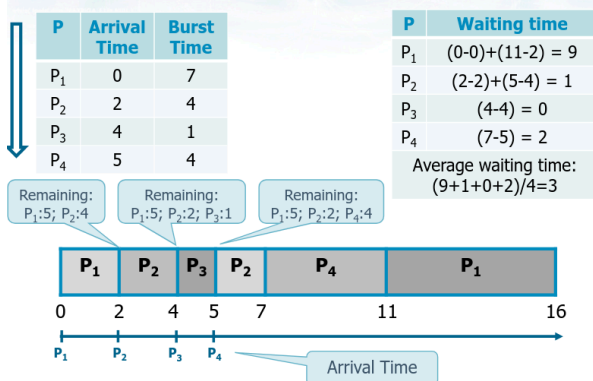


- Drawbacks
 - The average waiting time is relatively long
 - Substantial dependence of performance on the length of the quantum
 - Quantum long: RR degenerates into FCFS
 - Quantum short: too much context switching is performed, and switching/management times are very high (if compared with useful work)

SRTF

Version of SJF with preemption

- Algorithm
 - It proceeds with a scheduling of type SJF, but
 - if a task with smaller burst time (than the running one) is submitted, the CPU is preempted in favor of the new task
- Similar characteristics of the SJF scheduler

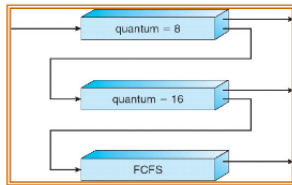


- Advantages
 - Short tasks are handled very quickly
 - Because the task with the least remaining time is executed and its remaining time can only decrease, context switching occurs only when a new process arrives
 - The overhead required by the algorithm is minimal
- Drawbacks
 - Like SJF, it requires an accurate estimate of the execution time
 - Like SJF suffers from starvation

MQS

Applied to situations where tasks can be classified into different groups (Foreground, background, system, etc.)

- Algorithm
 - The ready queue is divided into different queues
 - Each queue can be managed with its own scheduling algorithm
 - It can be modified to allow the transfer of tasks between the various queues
 - MQS with feedback



Considerazioni aggiuntive:

- Lo scheduler è un task che deve essere schedulato in maniera simile agli altri task
 - Nello scheduling senza prelazione
 - Lo scheduler è invocato ogni volta che un programma termina o lascia il controllo
 - Nello scheduling con prelazione
 - Lo scheduler è invocato periodicamente da un interrupt periodico della CPU
 - Gli altri task non possono prevenire questo procedimento
- Scheduling can be performed at the process or thread level
 - If the OS allows the use of threads, the scheduling is normally performed at the threads level (processes are not taken into account)
- Threads scheduling
 - The OS takes into account only T at kernel level, and it ignores T at user level (which are managed through a library)
 - As a consequence, the scheduling can be performed only for T at kernel level (if they exist)
- Scheduling for multiprocessors systems
 - All previous examples have been made assuming the existence of a single CPU
 - In the case of more than one CPU, load can be shared
 - The load balance is automatic for OS with waiting queues common to all processors
 - There are several schemes
 - Asymmetric multi-processing: a master processor distribute the load among slave processors
 - Symmetric multi-processing: each processor provides for its own scheduling
- Scheduling for real-time systems
 - They try to respond in real-time and within predefined deadline to events
 - Events (e.g., raise of a signal and subsequent interrupt) guide the scheduling
 - Latency is defined as the time elapsing between the occurrence of an event and its management
 - There are two types of real-time systems
 - Soft real-time
 - They give priority to critical processes, but do not guarantee response times (only probabilistic guarantees)
 - Hard real-time
 - The execution of the tasks is guaranteed within a maximum time limit (deadline)